

Using AI Agents

Tools, Connections and Multi-Agent Systems

Course Activities

- Tool calls create a usable outing comparison
- MCP connects a course calendar and task service
- Separate agents contribute to a shared menu proposal
- Guardrails and human approval control a fictional refund
- A coding harness repairs a small calculator

Browser Access

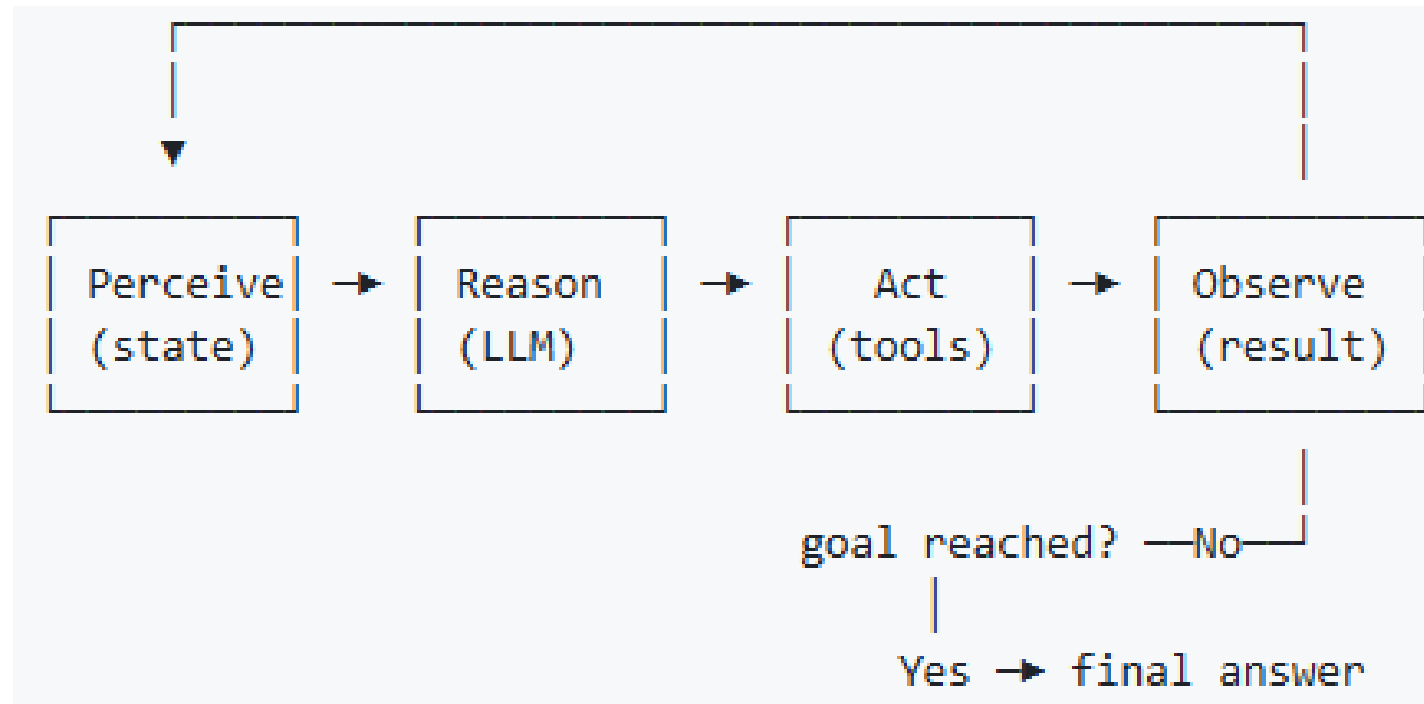
- Course code provides access to the hosted lab
- Browser activities require no software installation
- Fictional scenarios replace employer and customer data
- Downloads are optional
- The same activities support room and Teams participation

► Agents and Tools

- Model Context Protocol
 - Multi-Agent Systems
 - Guardrails and Human Review
 - Coding Agent Harnesses
- What is an Agent?
 - Tool Capabilities
 - Tool Descriptions
 - Tool Arguments
 - Browser Interaction
 - Code Execution
 - Output Formats
 - Service Performance Analysis
 - Tool Failures
 - Agents and Workflows
 - Context Within a Task

What is an Agent?

- An agent uses a model to select actions while pursuing a goal
- Tool results inform the next action or final response
- The host runs the tools and maintains the conversation



Tool Capabilities

Capability	Example operation
Browser	Navigate a site and complete a form
Code execution	Calculate values or run tests
File generation	Create Excel, PowerPoint or PDF output
Connected service	Create a calendar event or task
Image generation	Produce or edit a visual

- The host selects which tools are available to the model

Tool Descriptions

- **A tool definition describes its purpose and inputs :**
 - Name identifies the operation
 - Description explains when the operation is useful
 - Input schema describes required fields and allowed values
- **Overlapping descriptions can make tool selection ambiguous**
- **Narrow operations make behaviour easier to understand**

Tool Arguments

- The model requests a quote for eight people :

```
{  
  "name": "quote_activity", "arguments": {  
    "activity_id": "pottery", "people": 8  
  }  
}
```

- The function combines fixed and per-person costs :
 - Fixed fee of EUR 40
 - Eight participants at EUR 25 each
 - Returned total of EUR 240
- The returned value becomes context for the next model call

Browser Interaction

- **Browser tools expose page content and interaction controls :**
 - Links, fields and buttons
 - Screenshots or a structured page representation
- **An agent can navigate, enter values and inspect the result**
- **Page changes can invalidate earlier element references**
- **A browser session carries the access of its signed-in user**

Code Execution

- A code tool runs a calculation or program in an execution environment
- Code can transform CSV records and create charts
- Existing tests can guide an agent repairing a program
- Execution access requires limits :
 - Files and network destinations available to the process
 - Runtime, output size and resource consumption
- The browser mini harness uses restricted arithmetic interpretation

Output Formats

Output	Useful behaviour
Excel	Editable inputs and working formulas
PowerPoint	Editable slides for a chosen audience
HTML	Interactive controls in a browser
CSV	Data for another application

- **Generated output depends on the installed tool capability**
- **The course comparison works in the browser without Excel**

Service Performance Analysis

- **A service export supports an operational comparison :**
 - Response time grouped by region and reporting period
 - Record counts distinguish a broad pattern from a small sample
- **Calculation and presentation tools perform different work :**
 - Calculation produces totals and weighted averages
 - Chart generation makes regional differences visible
 - Presentation tools adapt the result for a management discussion
- **Service owners interpret causes and select an improvement trial**

Tool Failures

- A tool can return a timeout, invalid input or missing result
- The agent can choose a recovery step :
 - Retry a temporary failure within a limit
 - Correct the arguments
 - Ask for missing information
- A missing capability may require another tool or a smaller task
- Run limits prevent repeated unsuccessful calls

Agents and Workflows

- A workflow follows a predefined sequence and branches
- A model can choose intermediate steps in an agent loop
- One application can combine both approaches :
 - Fixed calculation inside a model-selected tool
 - Model interpretation inside a fixed processing pipeline
- Repeatable arithmetic belongs in the calculation tool
- The outing agent selects which alternatives to compare

Context Within a Task

- **Initial context defines the requested outcome :**
 - Goal, audience and constraints supplied in the request
 - Available tool definitions supplied by the host
- **Context grows as the task proceeds :**
 - Tool results supply prices, availability or project contents
 - Follow-up requests revise assumptions or output requirements
- **A change from eight to twelve participants affects the comparison :**
 - Per-person costs increase while fixed fees remain unchanged
 - The cheapest activity can change as group size changes

Summary : Agents and Tools

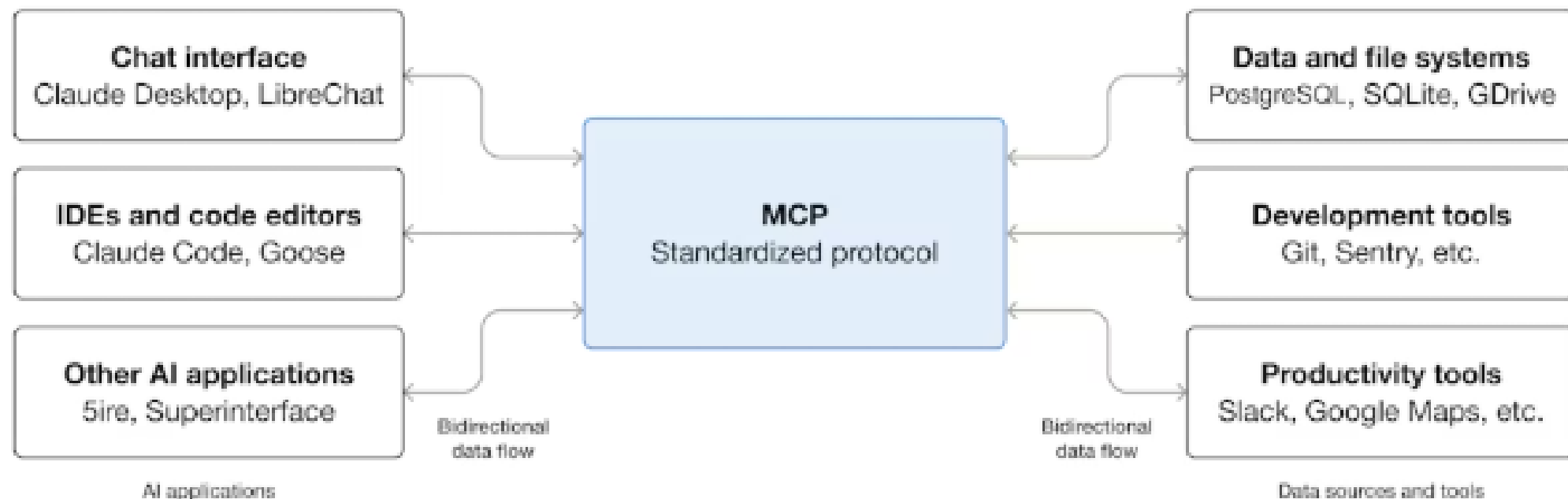
- Models choose among capabilities exposed by the host
- Tool descriptions and schemas guide selection and arguments
- Tools can calculate, create artifacts and interact with systems
- Browser and execution tools depend on their environment
- Useful context and follow-up requests shape the result
- The outing activity combines search, calculation and file generation

- Agents and Tools
- ▶ **Model Context Protocol**
- Multi-Agent Systems
- Guardrails and Human Review
- Coding Agent Harnesses

- What is MCP?
- MCP Architecture
- Server Capabilities
- Tool Discovery
- Local and Remote Servers
- Authentication and Scope
- Calendar Operations
- Account Management Connections
- Programme Coordination
- Disconnection and Revocation
- MCP Boundaries

What is MCP?

- Model Context Protocol standardises communication with external capabilities
- A server exposes operations and context to compatible clients
- Integration code can be reused across compatible hosts



MCP Architecture

Component	Course implementation
Host	Browser course application
Client	MCP connection inside the application
Server	Course calendar and task service
Model	Selects from the available tool definitions

- The client sends protocol requests to the server
- The server returns results to the host

Server Capabilities

- **Tools expose operations that a host can invoke :**
 - Read calendar entries or propose a new appointment
 - Input schemas describe the accepted arguments
- **Resources expose context that clients can retrieve :**
 - Identified content such as a document or service record
 - Host decides how retrieved content enters the conversation
- **Prompts expose reusable interaction templates :**
 - Server-provided starting points with optional arguments
 - Availability depends on the server's implementation
- **The course server exposes document access through tools**

Tool Discovery

- **Connection setup establishes protocol capabilities :**
 - Client and server exchange supported features
 - Authentication can fail before a model request begins
- **The client retrieves tool definitions from the server :**
 - Names and descriptions identify available operations
 - Schemas define required inputs and accepted values
- **The host chooses which definitions reach the model :**
 - A connection can expose more tools than one task needs
 - Tool discovery alone does not authorise every operation

Local and Remote Servers

Transport	Typical arrangement
Standard input/output	Host launches a local server process
Streamable HTTP	Client connects to a server endpoint

- Local servers can depend on installed runtimes
- Remote servers can serve multiple authorised clients
- The course uses a remote service over HTTPS
- Students connect through the browser host

Authentication and Scope

- **Credentials determine access available to a connection :**
 - Identity or sandbox associated with the request
 - Read and write operations allowed by the service
- **Course credentials separate responsibilities :**
 - Read access retrieves permitted fictional records
 - Proposal access prepares a task or calendar change
 - Separate review access permits the human decision
- **The server checks each operation against its access rules**
- **Approval behaviour belongs to the application and server**

Calendar Operations

- **Scheduling combines information from connected tools :**
 - Diary supplies the business-trip constraint
 - Calendar supplies existing appointments
- **A proposal specifies the intended change :**
 - Thursday 10 September 2026, at a free time after the trip
 - Twenty-minute planning review with a descriptive title
- **The host presents the proposed event for a decision :**
 - Accepted proposals can be applied to the course calendar
 - Rejected proposals leave existing appointments unchanged
- **Saved appointments remain available on a later connection**

Account Management Connections

- **An account task can span several connected systems :**
 - CRM supplies meeting notes and open customer questions
 - Task system assigns a follow-up to an account owner
 - Calendar reserves preparation time before the next meeting
- **Available tools determine the scope of action :**
 - A draft can be prepared without a messaging connection
 - Sending needs messaging access and user authorisation
- **The course Sales case creates one fictional follow-up task**

Programme Coordination

- **Programme coordination involves dependencies across teams :**
 - A telecom rollout can depend on engineering and commercial dates
 - Notes identify the blocked activity and responsible owner
- **An agent can prepare a targeted coordination action :**
 - Create an owner-assigned task for the unresolved dependency
 - Reserve a meeting when calendar access is available
- **Persistent systems keep actions available beyond the conversation**
- **Baseline changes require the designated programme authority**

Disconnection and Revocation

- **Disconnection ends the current host connection :**
 - Further tool calls need a new connection
 - Saved tasks and appointments remain on the server
- **Revocation changes access at the service :**
 - A revoked credential cannot authorise subsequent requests
 - Closing a browser does not revoke a copied credential
- **Local reset and remote state are separate :**
 - Resetting a browser exercise clears its local scenario
 - Earlier calendar appointments can affect a repeated demo

MCP Boundaries

- **MCP standardises communication between clients and servers :**
 - Discovery and calls follow protocol messages
 - Transport and feature support determine compatibility
- **Service implementation determines available actions :**
 - A calendar server can expose scheduling operations
 - A document server may provide retrieval without writes
- **Host and server policy determine execution controls :**
 - Credentials restrict access to operations and records
 - Approval controls decide when a person must intervene
- **Model choice and task context still affect the result**

Summary : MCP

- Hosts connect through clients to MCP servers
- Servers expose tools, resources or prompts
- Discovery makes available capabilities visible to the host
- Credentials and policy determine usable operations
- The course calendar demonstrates a persistent connected action
- Other systems can expose their own domain operations

- Agents and Tools
- Model Context Protocol
- ▶ **Multi-Agent Systems**
- Guardrails and Human Review
- Coding Agent Harnesses

- What is a Multi-Agent System?
- Agent Contexts
- Sequential Agents
- Parallel Agents
- Routing to a Specialist
- Supervisor Coordination
- Delegation Instructions
- Shared State
- Specialist Selection
- Service Improvement Teams
- Multi-Agent Tradeoffs

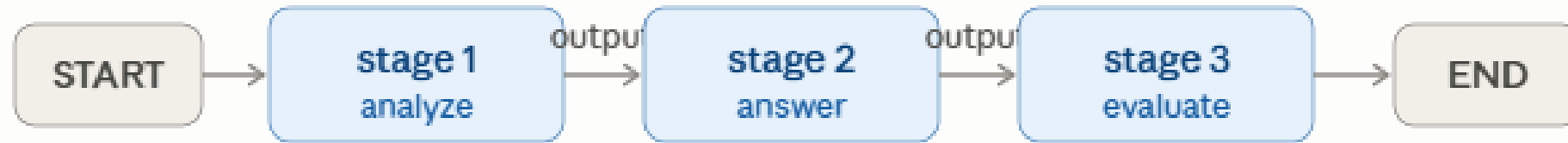
What is a Multi-Agent System?

- **Several agents contribute to a shared task :**
 - Each run receives its own instructions and context
 - Available tools can differ by responsibility
- **Coordination determines the flow of work :**
 - Assignments identify each agent's contribution
 - Returned results feed another stage or a final response
- **The course cafe team uses two specialists and a coordinator**
- **Separate model calls make the contributions independently visible**

Agent Contexts

- **Each agent receives the context supplied to its run :**
 - Specialist instructions focus attention on a smaller task
 - Tool results stay within that run unless explicitly shared
- **Context separation has practical consequences :**
 - Unrelated investigation does not fill the coordinator's context
 - A missing budget or audience can produce incompatible results
- **Returned summaries control what the next agent receives :**
 - Useful conclusions and unresolved questions
 - Enough detail to combine the work without repeating it

Sequential Agents



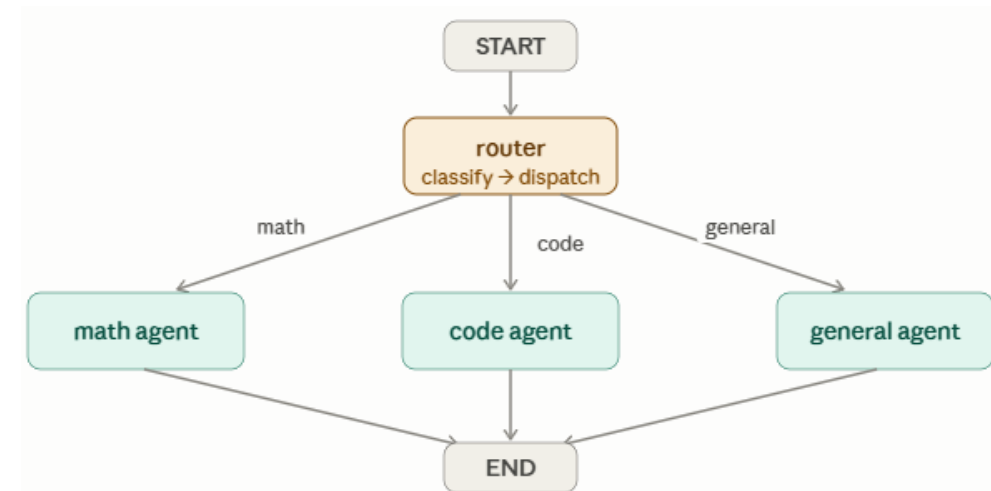
- Each stage receives the preceding result
- The order follows the task dependency
- A researcher can pass a shortlist to a writer
- Later stages wait for earlier stages to finish

Parallel Agents

- **Independent contributions can run concurrently :**
 - Creative specialist proposes a cafe concept and menu
 - Operations specialist calculates quantities and ingredient cost
- **Both specialists receive the shared constraints :**
 - Twenty visitors and a EUR 100 ingredient budget
 - Separate conversations prevent one proposal anchoring the other
- **A coordinator receives both results and resolves differences**
- **Elapsed time includes the specialist runs and final coordination**

Routing to a Specialist

- A router selects a suitable destination
- The selected specialist receives the task
- Unselected specialists perform no work
- Routing can depend on a rule or a model decision



Supervisor Coordination

- **A supervisor delegates tasks and receives results :**
 - Assignment names the contribution and relevant constraints
 - Returned information can change the next assignment
- **Coordination can include revision or further investigation :**
 - Request a cheaper menu after a budget overrun
 - Ask a specialist to resolve a missing assumption
- **The harness controls which delegation operations exist**
- **Turn and task limits bound repeated coordination**
- **The course cafe uses fixed orchestration; dynamic delegation is optional**

Delegation Instructions

- **A delegated task needs a defined contribution :**
 - Input context and constraints
 - Expected result and available tools
- **Shared constraints keep contributions compatible**
- **Output limits reduce unnecessary material passed back**
- **Agent roles should reflect separable work**

Shared State

- **Agents can exchange results through several mechanisms :**
 - Coordinator passes selected results between conversations
 - Shared files or services preserve joint progress
- **Concurrent writes create integration problems :**
 - Two edits can replace the same file contents
 - One agent can act on state another agent has already changed
- **Separate work areas reduce interference :**
 - Each agent produces an independently reviewable change
 - Integration combines accepted contributions and resolves conflicts

Specialist Selection

Specialist	Main concern
Creative	Menu concept and appeal
Operations	Portions and ingredient cost
Accessibility	Vegan choice and clear ordering

- Changing a role changes which questions receive attention
- The coordinator still needs to meet the overall task

Service Improvement Teams

- **A fictional service improvement task can be divided by responsibility :**
 - Analyst groups recurring service delays
 - Planner proposes a small operational trial
 - Coach prepares guidance for the service team
- **A coordinator combines the proposed changes**
- **Quality and service owners retain operational decisions**

Multi-Agent Tradeoffs

- **Parallel work benefits tasks with independent contributions :**
 - Separate investigations can progress at the same time
 - Focused contexts reduce unrelated information
- **Coordination adds work of its own :**
 - Extra model calls consume tokens
 - Integration takes time and can require another revision
 - Agents can repeat work or share a mistaken assumption
- **A solo run provides a task-specific comparison**
- **The cafe interface reports elapsed time and available token usage**

Summary : Multi-Agent Systems

- Multiple agents can divide work by task or role
- Sequential stages pass results forward
- Parallel agents work on independent contributions
- Routers select a destination and supervisors coordinate delegation
- Shared context and integration determine whether contributions fit
- Extra agents are useful when the division of work earns its cost

- Agents and Tools
- Model Context Protocol
- Multi-Agent Systems
- ▶ **Guardrails and Human Review**
- Coding Agent Harnesses

- What are Guardrails?
- Instructions and Rules
- Tool Permissions
- Argument Validation
- Cumulative Limits
- Human in the Loop
- Approval Content
- Execution Sandboxes
- Untrusted Content
- Mobile Release Controls
- Recovery After Interruption

What are Guardrails?

- Guardrails constrain an agent's inputs, actions or outputs
- Controls can operate outside the model :
 - Tool allowlists and argument validation
 - Spending limits and execution boundaries
- Instructions influence model behaviour
- Enforced controls reject disallowed operations

Instructions and Rules

Control	Example
Model instruction	Ask before proposing a refund
Tool validation	Reject an amount above EUR 50
Human decision	Accept or reject the pending proposal

- **An instruction does not replace tool-side validation**
- **The course model has no approval tool**

Tool Permissions

- **The host chooses the operations an agent can request :**
 - Read-only tools inspect permitted information
 - Write tools expose specific changes to records or files
- **Permission boundaries can separate responsibilities :**
 - Proposal access lets the agent prepare an operation
 - Review access belongs to a separate human control
- **A restricted tool list reduces available actions**
- **Credentials and tool-side checks enforce the service boundary**

Argument Validation

- **Tool code validates values before applying an operation :**
 - Known record identifiers
 - Amount and quantity ranges
 - Required fields and expected types
- **Well-formed JSON can still contain an invalid request**
- **An error result allows the agent to correct its arguments**

Cumulative Limits

- The fictional shop permits EUR 50 in total refunds per order

Request	Refunded so far	Result
Propose EUR 40	EUR 0	Available for review
Approve EUR 40	EUR 0	Ledger records EUR 40
Propose another EUR 40	EUR 40	Rejected; EUR 10 remains

- **The limit applies across repeated requests :**
 - Splitting an amount does not increase the allowance
 - Reset clears this isolated course scenario

Human in the Loop

- **Human intervention occurs at a defined decision point :**
 - Agent prepares a concrete operation and its arguments
 - The pending operation waits before changing the destination
- **The refund controls implement two outcomes :**
 - Approval applies the displayed proposal to the ledger
 - Rejection leaves the ledger unchanged
- **A rejected proposal can be revised in a follow-up request**
- **Routine retrieval can continue without repeated approval prompts**

Approval Content

- **A review control displays the pending operation :**
 - Destination and record identifier
 - Amount or requested change
 - Reason supplied for the action
- **Approval refers to the displayed proposal**
- **Stale or repeated approval requests are rejected**

Execution Sandboxes

- **A sandbox restricts the environment reached by an operation :**
 - Files and directories available to the process
 - Process execution and outbound network access
- **Permissions and isolation serve different purposes :**
 - Permission decides whether an operation may proceed
 - Isolation limits the reach of an allowed operation
- **The browser calculator uses a restricted arithmetic interpreter :**
 - Allows named values, arithmetic and conditional expressions
 - Rejects imports, function calls and arbitrary code execution

Untrusted Content

- **Retrieved content can contain instructions addressed to an agent :**
 - A web page can request an unrelated upload
 - A document can imitate a user or system message
- **The source remains task data with no authority to change the goal**
- **Execution controls limit consequences of a mistaken interpretation :**
 - Restricted tools remove unnecessary actions
 - Destination rules limit outbound access
 - Human review can intercept a proposed external change

Mobile Release Controls

- **A fictional mobile release task separates preparation from release :**
 - Agent generates test ideas or summarises failures
 - Test tools execute the available checks
 - Release control waits for the designated decision
- **Passing tests does not grant deployment access**
- **iOS and Android can require different test environments**

Recovery After Interruption

- **An interrupted request can leave operation status uncertain :**
 - The service may have saved the change before the connection failed
 - Repeating the request can create a duplicate without protection
- **Recovery starts from the destination's current state :**
 - Retrieve the saved task, appointment or ledger entry
 - Resume a pending proposal if no change was applied
- **Stable operation identifiers can prevent duplicate application**
- **Time and turn limits stop unproductive retries**

Summary : Guardrails

- Instructions and enforced rules operate at different layers
- Tool validation rejects invalid arguments
- Cumulative limits cover repeated operations
- Human review controls selected decisions
- Sandboxes restrict execution reach
- Recovery checks the current state before repeating an action

- Agents and Tools
- Model Context Protocol
- Multi-Agent Systems
- Guardrails and Human Review
- ▶ **Coding Agent Harnesses**

- What is a Harness?
- The Coding Agent Loop
- Code Changes
- Project Instructions
- Reusable Skills
- MCP in a Harness
- Subagents in a Harness
- Codex
- Claude Code
- Antigravity CLI
- Harness Comparison

What is a Harness?

- **A harness supplies the application around the model :**
 - Maintains conversation and project context
 - Executes requested tools and returns their results
- **A coding harness exposes project operations :**
 - Search and read files, edit code and run checks
 - Connect external services when configured
- **Application controls govern the working environment :**
 - Permissions and isolation restrict tool execution
 - Progress views and interruption controls support supervision

The Coding Agent Loop

1. The model selects an operation from the available tools
 2. The harness executes the operation and returns its result
 3. The model uses the result to choose the next step
- **A failed check can lead to another edit :**
 - Test output identifies the observed mismatch
 - The agent changes the implementation and runs checks again
 - **The loop stops on completion, interruption or an execution limit**
 - **The final response reports the result reached by the run**

Code Changes

- The calculator request changes the implemented operation :

```
# Starter implementation
return price + quantity

# Requested implementation
return price * quantity
```

- Prepared checks distinguish the two behaviours :
 - Price 10 and quantity 2 produce 20 instead of 12
 - Zero quantity produces a zero total
- A later request introduces a 10% discount from EUR 50

Project Instructions

- Project instructions provide persistent guidance
- Useful instructions identify project conventions :
 - Function names or interfaces to preserve
 - Commands used to run checks
 - Dependency constraints
- File discovery conventions differ between products
- An explicit request can name a shared instruction file

Reusable Skills

- **A skill packages guidance for a recurring task :**
 - Instructions describe the procedure and expected output
 - Supporting templates or scripts can perform specialised work
- **A presentation skill can define production conventions :**
 - Slide structure, typography and reusable layouts
 - Rendering and inspection before delivery
- **The host determines discovery and invocation**
- **Installed skills do not grant additional account permissions**

MCP in a Harness

- **A harness can connect external capabilities through MCP :**
 - Server tools join the operations available to a task
 - Discovery exposes descriptions and input schemas
- **A coding task can span a project and a connected service :**
 - Retrieve an issue containing the requested behaviour
 - Edit the implementation and run the project's checks
 - Prepare an issue update with the result
- **Credentials and host policy control service access**
- **Connection configuration differs between products**

Subagents in a Harness

- **A subagent receives a focused task in another context :**
 - Role instructions define the expected contribution
 - Tool access can be restricted to that responsibility
- **Delegation separates work that can proceed independently :**
 - Investigate an unfamiliar component
 - Review existing tests while another agent prepares a change
- **The parent receives the returned contribution**
- **Shared project edits still require coordination and integration**

Codex

- **OpenAI coding agent available through a CLI and other interfaces**
- **Project work uses executable tools :**
 - Inspect files and project instructions
 - Edit an implementation and run available commands
- **Skills and MCP connections extend task capabilities**
- **The calculator comparison exposes the harness controls :**
 - Progress through the requested repair
 - Permission settings and the resulting changes
 - Check output returned to the agent

Claude Code

- **Anthropic coding agent with terminal and other interfaces**
- **Project context guides the agent loop :**
 - Instructions and relevant files inform tool selection
 - Command results guide subsequent changes
- **Subagents support focused contributions :**
 - Specialised instructions and independently scoped context
 - Tool access configured for the assigned role
- **Skills and MCP connections extend the available workflow**
- **The comparison uses the same calculator request and starter**

Antigravity CLI

- **Antigravity CLI is Google's terminal agent interface**
- **Background subagents support concurrent work :**
 - Separate sessions investigate or implement assigned tasks
 - The agent panel displays progress and pending approvals
- **Extensions package reusable capabilities :**
 - Plugins can include skills, agents and MCP definitions
 - The MCP panel manages connected server tools
- **Permission settings and terminal sandboxing control execution**
- **Trainer comparison uses a fresh calculator starter**

Harness Comparison

Comparison point	Observation
Task interaction	Request, correction and interruption
Project context	Instructions and files loaded
Tool access	Available commands and connections
Human control	Permissions and proposed changes
Result	Repaired calculator and running checks

- Record product version, model and relevant settings

Summary : Harnesses

- The harness manages tools and context around the model
- Coding tools connect instructions to project changes
- Project instructions preserve conventions across tasks
- Skills and MCP provide different forms of extension
- Subagents can separate focused parts of the work
- The browser mini harness supports practice without installation